

Guide pratique de mise a jour d'un bot Discord auto-heberge

Ubuntu Server + Docker Compose + SQLite

Procedure reconstruite a partir d'un deploiement reel et valide

Objectif - Pouvoir mettre a jour le bot sans connaitre Linux par coeur, comprendre chaque commande executee, sauvegarder la base SQLite avant les operations sensibles, appliquer une migration de schema proprement, puis inspecter ou modifier des donnees avec SELECT, INSERT et UPDATE.

Version 1.0 - 7 septembre 2026

1. Comment utiliser ce guide

Ce document est volontairement pedagogique. Il ne se contente pas de donner des commandes a copier-coller : il explique ce que fait chaque commande, pourquoi elle est lancee a ce moment precis, ce qu'il faut verifier dans son resultat, et quand il faut s'arreter au lieu de continuer.

Principe de securite - Une mise a jour de code, une migration de schema et une correction de donnees sont trois operations differentes. On les traite separement pour pouvoir diagnostiquer et revenir en arriere proprement.

1.1 Placeholders anonymises

Les identifiants personnels, adresses IP et noms de machine ne sont pas conserves. Les commandes utilisent les marqueurs suivants :

Marqueur	Signification
<SERVER_USER>	Compte Linux utilise pour se connecter au serveur.
<SERVER_ID>	Nom logique du serveur dans les chemins /srv.
<PROJECT>	Nom du projet / service.
<SERVICE>	Nom du service Docker Compose, par exemple clavier.
<DEPLOY_BRANCH>	Branche Git actuellement deployee.
<DB_PATH>	Chemin complet vers la base SQLite de production.
<BACKUP_DIR>	Dossier ou sont stockees les sauvegardes SQLite.

1.2 Arborescence de reference

```
/srv/<SERVER_ID>/
├─ apps/<PROJECT>/      # clone Git, code source
├─ stacks/<PROJECT>/    # compose.yaml de production
├─ data/<PROJECT>/      # donnees persistantes, dont SQLite
└─ backups/             # sauvegardes manuelles avant operations sensibles
```

Dans les exemples, <DB_PATH> correspond typiquement a :

```
/srv/<SERVER_ID>/data/<PROJECT>/<PROJECT>.db
```

2. Le workflow valide : ce que nous avons reellement fait

Pour le hotfix ayant introduit une migration de base, la sequence correcte a ete la suivante. Cette chronologie est importante : chaque etape reduit le risque de la suivante.

1. Verifier la branche de deploiement et le dernier commit present sur le serveur.

2. Recuperer les nouveaux commits avec Git sans autoriser un merge implicite non prevu.
3. Construire la nouvelle image Docker a partir du nouveau code.
4. Arrêter le bot avant l'operation sensible sur SQLite.
5. Sauvegarder la base V4 avant toute migration.
6. Recreer le conteneur avec la nouvelle image et verifier que le code execute attend bien le schema V5.
7. Lancer la migration avec la commande applicative, pas avec un ALTER TABLE manuel.
8. Verifier que la base est maintenant V5 et que la colonne a bien ete renommee.
9. Arrêter de nouveau le bot avant une correction manuelle de donnees.
10. Sauvegarder la base V5 avant l'UPDATE de donnees.
11. Faire l'UPDATE dans une transaction, controler le nombre de lignes modifiees et l'integrite SQLite.
12. Redemarrer le bot, lire les logs et effectuer les tests fonctionnels.

Pourquoi deux sauvegardes ? - La sauvegarde pre-migration permet de revenir a la V4 complete. La sauvegarde post-migration permet de conserver la V5 correcte et de revenir uniquement sur une correction de donnees. Cela evite de devoir annuler une migration de schema simplement parce qu'un UPDATE de contenu etait mauvais.

3. Les commandes Linux utilisees, expliquees

Une commande Linux est composee d'un programme, d'options et parfois d'arguments. Exemple :

```
sudo docker compose logs --tail=100 <SERVICE>
```

Lecture de gauche a droite :

- `sudo` : execute la commande suivante avec les privileges administrateur. Docker et les fichiers proteges de `/srv` peuvent l'exiger.
- `docker` : lance le client Docker.
- `compose` : utilise le fichier `compose.yaml` du repertoire courant.
- `logs` : demande les journaux du service.
- `--tail=100` : limite l'affichage aux 100 dernieres lignes.
- `<SERVICE>` : nom du service a inspecter.

3.1 Se placer dans le bon dossier : `cd`

```
cd /srv/<SERVER_ID>/apps/<PROJECT>
```

`cd` signifie change directory. Beaucoup de commandes Git ou Docker Compose utilisent le repertoire courant. Se placer dans le mauvais dossier peut donc faire agir la commande sur le mauvais projet ou le mauvais `compose.yaml`.

3.2 Verifier l'etat des fichiers : `git status`

```
git status
```

Cette commande ne modifie rien. Elle indique notamment si des fichiers locaux ont ete modifies, ajoutes ou supprimes. Avant un pull de production, on veut idealement un working tree clean. Des modifications locales non commitees sur le serveur sont un signal d'alerte.

3.3 Connaitre la branche courante

```
git branch --show-current
```

Affiche uniquement le nom de la branche active. Cela evite de mettre a jour main alors que la production suit une branche de deploiement, ou l'inverse.

3.4 Identifier exactement le commit deploye

```
git log -1 --oneline
```

`git log` affiche l'historique. `-1` limite a un seul commit. `--oneline` condense le hash court et le message sur une ligne. Cette commande permet de relier un etat de production a un commit precis.

3.5 Recuperer la difference distante : `git pull --ff-only`

```
git pull --ff-only origin <DEPLOY_BRANCH>
```

`git pull` recupere les commits distants puis avance la branche locale. `--ff-only` impose un fast-forward : si Git devrait creer un commit de merge, la commande s'arrete au lieu de prendre une decision a votre place. Sur un serveur de production, ce comportement conservateur est souhaitable.

A retenir - Un serveur de production ne devrait pas devenir un poste de developpement. On y recupere des commits deja testes ; on evite d'editer le code directement sur place.

3.6 Chercher une information dans le code : grep

```
grep -n "CURRENT_SCHEMA_VERSION" src/claviger/database/schema.py
```

grep recherche du texte. -n ajoute le numero de ligne. Ici, l'objectif etait de verifier quelle version de schema le code source declarait comme version cible.

```
grep -n -A8 -B2 "5:" src/claviger/database/schema.py
```

-A8 affiche 8 lignes After le match et -B2 affiche 2 lignes Before. C'est pratique pour voir le bloc complet d'une migration sans ouvrir un editeur.

4. Construire et remplacer le conteneur Docker

4.1 Construire la nouvelle image

```
cd /srv/<SERVER_ID>/stacks/<PROJECT>
sudo docker compose build
```

Cette commande lit compose.yaml, puis le Dockerfile du projet. Elle fabrique une nouvelle image contenant le code actuellement present dans le clone Git. Le conteneur qui tourne deja n'est pas remplace pendant la construction : on peut donc preparer l'image avant la coupure.

Image != conteneur - L'image est le modele construit. Le conteneur est l'instance en cours d'execution. Mettre le code source a jour ne suffit pas : il faut reconstruire l'image puis recreer le conteneur.

4.2 Rebuild sans cache quand un doute subsiste

```
sudo docker compose build --no-cache <SERVICE>
```

--no-cache force Docker a refaire toutes les etapes du Dockerfile. Ce n'est pas necessaire a chaque mise a jour, mais c'est un bon outil de diagnostic quand le conteneur semble executer une ancienne version du code.

4.3 Arrêter proprement le service

```
sudo docker compose stop <SERVICE>
```

stop demande au conteneur de s'arreter sans supprimer sa definition. On l'utilise avant une sauvegarde ou une ecriture manuelle dans SQLite afin qu'aucun processus du bot ne tente d'ecrire en meme temps.

4.4 Recreer le conteneur avec la nouvelle image

```
sudo docker compose up -d --force-recreate <SERVICE>
```

up lance le service. -d signifie detached : la commande rend la main au terminal. --force-recreate oblige Compose a remplacer le conteneur meme si sa configuration parait identique. Les volumes / bind mounts persistent : la base SQLite situee dans /srv/.../data n'est pas supprimee avec le conteneur.

4.5 Verifier que le conteneur tourne

```
sudo docker compose ps
```

Affiche l'etat des services du compose : running, exited, etc. Cette commande permet de distinguer un bot qui ne s'est pas lance d'un simple delai d'affichage cote Discord.

4.6 Lire les logs

```
sudo docker compose logs --tail=100 <SERVICE>
```

Les logs permettent de reperer rapidement une erreur de configuration, un probleme de permission, une exception Python ou un echec de connexion Discord. Utiliser --tail evite d'imprimer tout l'historique.

4.7 Interroger le code vraiment execute dans le conteneur

```
sudo docker exec <SERVICE> python -c "from claviger.database.schema import CURRENT_SCHEMA_VERSION; print(CURRENT_SCHEMA_VERSION)"
```

docker exec lance une commande a l'interieur du conteneur deja demarre. C'est le controle le plus fiable lorsque le code source sur l'hote dit une chose mais que Discord en affiche une autre. Ici, on demande directement au Python du conteneur quelle version de schema il connait.

5. Sauvegarder SQLite avant une operation sensible

La base de production doit etre sauvegardee avant une migration de schema et avant une correction manuelle de donnees. Plutot qu'un simple cp pendant que SQLite peut encore etre utilise, on emploie l'API de backup fournie par sqlite3.

```
sudo python3 - <<'PY'
import sqlite3

src = sqlite3.connect("<DB_PATH>")
dst = sqlite3.connect("<BACKUP_DIR>/<PROJECT>-pre-operation.db")

src.backup(dst)

print("Integrity:", dst.execute("PRAGMA integrity_check").fetchone()[0])
print("Schema:", dst.execute("PRAGMA user_version").fetchone()[0])

dst.close()
src.close()
PY
```

5.1 Comprendre la syntaxe python3 - <<'PY'

python3 - signifie : lance Python et lis le programme depuis l'entree standard. La construction <<'PY' est un here-document du shell : tout ce qui suit jusqu'a la ligne finale PY est fourni a Python comme programme. Cela permet d'executer un petit script de maintenance directement depuis le terminal, sans creer un fichier .py permanent.

5.2 Ce que fait chaque ligne du backup

Instruction	Role
sqlite3.connect("<DB_PATH>")	Ouvre la base source de production.
sqlite3.connect("<BACKUP_DIR>/...")	Cree ou ouvre le fichier de sauvegarde destination.
src.backup(dst)	Utilise le mecanisme de backup SQLite pour recopier une image coherente de la base.
PRAGMA integrity_check	Demande a SQLite de verifier la coherence interne des pages, index et structures.
PRAGMA user_version	Lit le numero de version de schema gere par l'application.
fetchone()[0]	Recupere la premiere ligne puis la premiere colonne du resultat SQL.
close()	Ferme proprement les connexions.

Resultat attendu - Integrity: ok. Le numero Schema doit correspondre a l'etat que vous pensez sauvegarder, par exemple 4 avant migration puis 5 apres migration.

6. Migration de schema : laisser l'application la piloter

Une migration de schema change la structure de la base : ajout ou suppression de colonne, renommage, creation de table, index, etc. Lorsque le projet possede son propre moteur de migrations, il ne faut pas executer manuellement l'ALTER TABLE correspondant en production.

Pourquoi ne pas faire ALTER TABLE a la main ? - Parce que la structure pourrait devenir V5 alors que PRAGMA user_version resterait a 4. L'application tenterait alors de rejouer la migration, avec un risque d'erreur ou d'etat incoherent.

6.1 Verifier la version actuelle et la version attendue

Dans le bot, la commande de statut doit montrer une base encore en ancienne version mais un code qui attend la nouvelle version. Exemple :

```
/claviger database status
Version actuelle : 4
Version attendue : 5
```

Si la version attendue reste a 4 apres le deploiement du nouveau code, le probleme se situe dans le code cible ou dans l'image Docker, pas dans la base. Il faut s'arreter et diagnostiquer avant de migrer.

6.2 Lancer la migration

```
/claviger database migrate
```

La commande applicative execute les scripts de migration dans l'ordre prevu puis met a jour la version de schema. Dans le cas valide, la migration 5 renommait une colonne dans guild_settings.

6.3 Confirmer la migration

```
/claviger database status
```

Le bon etat est : version actuelle = version attendue. Ensuite, on peut verifier directement SQLite pour confirmer le numero de schema et les colonnes presentes.

```
sudo python3 - <<'PY'
import sqlite3

db = sqlite3.connect("<DB_PATH>")
print("Schema:", db.execute("PRAGMA user_version").fetchone()[0])

columns = [row[1] for row in db.execute("PRAGMA table_info(guild_settings)")]
print(columns)

db.close()
PY
```

PRAGMA table_info(table) renvoie une ligne par colonne. row[1] correspond au nom de la colonne. On peut ainsi verifier qu'un ancien nom a disparu et que le nouveau existe.

7. Travailler avec SQLite depuis Ubuntu sans apprendre un nouvel outil

Il n'est pas obligatoire d'installer le programme sqlite3 en ligne de commande. Python 3 est present sur Ubuntu Server et fournit le module standard sqlite3. Les exemples ci-dessous utilisent donc la meme methode que les sauvegardes et les controles deja effectues.

Regle principale - Pour une simple lecture, le bot peut souvent continuer a tourner. Pour une ecriture manuelle INSERT/UPDATE/DELETE sur la base de production, arretez le bot avant l'operation et sauvegardez la base.

7.1 SELECT : lire des donnees

```
sudo python3 - <<'PY'
import sqlite3

db = sqlite3.connect("<DB_PATH>")

rows = db.execute("""
    SELECT column_a, column_b
    FROM table_name
    WHERE id = ?
""", (123,)).fetchall()

for row in rows:
    print(row)

db.close()
PY
```

Lecture SQL, morceau par morceau :

- **SELECT column_a, column_b** : choisit les colonnes a retourner. **SELECT *** retourne toutes les colonnes, mais est moins explicite.
- **FROM table_name** : indique la table lue.
- **WHERE id = ?** : ne conserve que les lignes dont id correspond au parametre fourni.
- **?** : placeholder parametre. Python remplace proprement cette valeur avec le tuple (123,).
- **fetchall()** : recupere toutes les lignes du resultat dans une liste.
- **for row in rows** : parcourt les lignes pour les afficher.

Attention au WHERE - Un SELECT sans WHERE lit toutes les lignes. Ce n'est pas dangereux pour les donnees, mais peut afficher beaucoup d'informations. Pour UPDATE ou DELETE, oublier WHERE peut modifier ou supprimer toute la table.

7.2 SELECT COUNT(*) : compter sans afficher le contenu

```
count = db.execute("SELECT COUNT(*) FROM table_name WHERE enabled = ?", (1,)).fetchone()[0]
print("Nombre de lignes:", count)
```

COUNT(*) demande a SQLite de compter les lignes correspondant au filtre. C'est ideal pour auditer une migration sans afficher des donnees sensibles.

7.3 INSERT : ajouter une ligne

```
sudo python3 - <<'PY'
import sqlite3

db = sqlite3.connect("<DB_PATH>")

try:
    db.execute("BEGIN IMMEDIATE")

    cursor = db.execute("""
        INSERT INTO table_name (column_a, column_b)
        VALUES (?, ?)
        """, ("value_a", "value_b"))

    print("Lignes ajoutees:", cursor.rowcount)
    db.commit()
    print("COMMIT effectue.")

except Exception:
    db.rollback()
    print("ROLLBACK effectue.")
    raise

finally:
    db.close()

PY
```

INSERT INTO table_name (...) indique la table et les colonnes renseignees. VALUES (?, ?) fournit une valeur pour chaque colonne. Le tuple Python associe les vraies valeurs aux placeholders dans le meme ordre.

7.4 UPDATE : modifier une ou plusieurs lignes

```
sudo python3 - <<'PY'
import sqlite3

db = sqlite3.connect("<DB_PATH>")

try:
    db.execute("PRAGMA foreign_keys = ON")
    db.execute("BEGIN IMMEDIATE")

    cursor = db.execute("""
        UPDATE table_name
        SET column_b = ?
        WHERE id = ?
        AND column_b = ?
        """, ("new_value", 123, "expected_old_value"))

    print("Lignes modifiees:", cursor.rowcount)
```

```

    if cursor.rowcount != 1:
        raise RuntimeError("Une seule ligne etait attendue")

    integrity = db.execute("PRAGMA integrity_check").fetchone()[0]
    foreign_keys = db.execute("PRAGMA foreign_key_check").fetchall()

    print("Integrity:", integrity)
    print("Foreign keys:", "ok" if not foreign_keys else foreign_keys)

    if integrity != "ok" or foreign_keys:
        raise RuntimeError("Controle SQLite echoue")

    db.commit()
    print("COMMIT effectue.")

except Exception:
    db.rollback()
    print("ROLLBACK effectue.")
    raise

finally:
    db.close()
PY

```

Le filtre WHERE contient volontairement l'identifiant ET l'ancienne valeur attendue. Cela rend l'operation defensive : si la base n'est pas dans l'etat prevu, rowcount vaut 0 et le script annule au lieu d'ecraser silencieusement une valeur inattendue.

8. Transactions : COMMIT, ROLLBACK et BEGIN IMMEDIATE

Une transaction regroupe plusieurs operations en une seule unite logique. Tant que commit() n'a pas ete appele, les modifications peuvent etre annulees avec rollback().

Element	Role
BEGIN IMMEDIATE	Demarre une transaction d'ecriture et reserve le droit d'ecrire. Utile pendant une maintenance controlee.
db.execute(...)	Execute les SELECT/INSERT/UPDATE necessaires.
cursor.rowcount	Indique combien de lignes ont ete affectees par INSERT/UPDATE/DELETE.
db.commit()	Valide definitivement les modifications.
db.rollback()	Annule les modifications de la transaction si une erreur survient.
finally: db.close()	Ferme la connexion qu'il y ait succes ou echec.

Pour une correction de production, un script qui dit seulement UPDATE puis commit sans verifier rowcount est trop confiant. Les controles doivent faire partie de la transaction.

9. Les PRAGMA SQLite les plus utiles en maintenance

Commande	Ce qu'elle remonte
PRAGMA user_version	Numero de version du schema gere par l'application.
PRAGMA table_info(table_name)	Liste des colonnes, types, contraintes et ordre de la table.
PRAGMA integrity_check	Controle de coherence interne de la base. Le resultat normal est ok.
PRAGMA foreign_key_check	Liste les violations de cles etrangeres. Une liste vide signifie aucun probleme detecte.
PRAGMA foreign_keys = ON	Active l'application des contraintes de cles etrangeres pour la connexion courante.

```
sudo python3 - <<'PY'
import sqlite3

db = sqlite3.connect("<DB_PATH>")
```

```
print("Schema:", db.execute("PRAGMA user_version").fetchone()[0])
print("Integrity:", db.execute("PRAGMA integrity_check").fetchone()[0])
print("Foreign keys:", db.execute("PRAGMA foreign_key_check").fetchall())

for row in db.execute("PRAGMA table_info(guild_settings)"):
    print(row)

db.close()
PY
```

Cette commande est un bon diagnostic generique : elle ne modifie rien et permet de confirmer la version, l'integrite, les cles etrangeres et la structure d'une table.

10. Decouvrir la base quand on ne connait pas encore son schema

10.1 Lister les tables

```
rows = db.execute("""
    SELECT name
    FROM sqlite_master
    WHERE type = 'table'
    ORDER BY name
    """).fetchall()

for row in rows:
    print(row[0])
```

sqlite_master est le catalogue interne de SQLite. type = 'table' ne garde que les tables. ORDER BY name trie les resultats alphabetiquement.

10.2 Voir les colonnes d'une table

```
for row in db.execute("PRAGMA table_info(table_name)"):
    print(row)
```

10.3 Compter les lignes

```
print(db.execute("SELECT COUNT(*) FROM table_name").fetchone()[0])
```

10.4 Afficher un petit echantillon

```
rows = db.execute("SELECT * FROM table_name LIMIT 10").fetchall()
for row in rows:
    print(row)
```

LIMIT 10 evite d'imprimer toute la table. Pour une base contenant des informations sensibles, selectionnez plutot uniquement les colonnes dont vous avez besoin.

11. Etude de cas : pourquoi la migration V5 n'etait pas visible au debut

Le code contenait bien un bloc de migration numero 5, mais la constante `CURRENT_SCHEMA_VERSION` etait encore fixee a 4. Cela illustre une distinction essentielle entre "une migration existe dans le fichier" et "le moteur considere cette migration comme cible".

```
MIGRATIONS = {
    ...
    5: (
        """
        ALTER TABLE guild_settings
        RENAME COLUMN old_column_name
        TO new_column_name
        """,
    ),
}

CURRENT_SCHEMA_VERSION = 5
```

Tant que `CURRENT_SCHEMA_VERSION` restait a 4, /claviger database status annoncait version attendue 4 et le moteur n'avait aucune raison d'appliquer la migration 5.

Le diagnostic a utilise trois niveaux de verification :

1. grep sur le code source de l'hote pour confirmer que la migration 5 existait.
2. grep sur `CURRENT_SCHEMA_VERSION` pour constater que la cible restait 4.
3. docker exec apres rebuild pour demander au Python reellement execute dans le conteneur quelle version il connaissait.

Lecon - Quand la production ne reflète pas le code attendu, comparez toujours trois états : depot Git sur le serveur, image/conteneur Docker, puis base SQLite. Ils peuvent être à des versions différentes.

12. Trois types de mise a jour a distinguer

Type	Procedure minimale
Code uniquement	Pull -> build -> force-recreate -> logs -> tests.
Code + migration de schema	Pull -> build -> stop -> backup -> recreate -> verifier cible -> migrate -> backup -> tests.
Correction de donnees	Stop -> backup -> SELECT de controle -> transaction UPDATE/INSERT -> controles -> commit -> restart.

En cas de doute sur le type de release, choisissez le chemin le plus conservateur : sauvegarde avant de toucher a la persistance.

13. Procedure standard a suivre lors d'une prochaine mise a jour

Phase A - Avant la coupure

```
cd /srv/<SERVER_ID>/apps/<PROJECT>
git status
git branch --show-current
git log -1 --oneline
git pull --ff-only origin <DEPLOY_BRANCH>
git log -1 --oneline

cd /srv/<SERVER_ID>/stacks/<PROJECT>
sudo docker compose build
```

But : verifier ce qui tourne, recuperer exactement la branche voulue, puis preparer l'image avant l'indisponibilite du bot.

Phase B - Si la release touche au schema

```
sudo docker compose stop <SERVICE>
# Faire la sauvegarde SQLite pre-migration
sudo docker compose up -d --force-recreate <SERVICE>
sudo docker compose logs --tail=100 <SERVICE>
# Verifier version actuelle / version attendue
# Lancer la migration applicative
# Re-verifier version actuelle / version attendue
```

But : conserver une copie de la Vn, executer la nouvelle application, laisser son moteur de migration transformer la DB en Vn+1, puis confirmer que le schema est coherent.

Phase C - Si une donnee doit etre corrige

```
sudo docker compose stop <SERVICE>
# Faire une seconde sauvegarde, maintenant au nouveau schema
# SELECT de controle
# UPDATE / INSERT dans une transaction
# Verifier rowcount, integrity_check et foreign_key_check
# COMMIT uniquement si tout est correct
sudo docker compose up -d <SERVICE>
```

But : separer le changement structurel du changement de contenu. Si la correction de donnee est mauvaise, la sauvegarde post-migration permet de restaurer la base sans revenir a l'ancien schema.

Phase D - Validation

```
sudo docker compose ps
sudo docker compose logs --tail=100 <SERVICE>
```

Puis utiliser les commandes fonctionnelles du bot : statut DB, scan des roles, synchronisation des catalogues et parcours utilisateur concerne par le hotfix.

14. Restaurer une sauvegarde SQLite en cas de probleme

Une sauvegarde n'a de valeur que si l'on sait comment la restaurer. La restauration doit etre effectuee
bot arrete.

```
cd /srv/<SERVER_ID>/stacks/<PROJECT>
sudo docker compose stop <SERVICE>

sudo cp <BACKUP_DIR>/<BACKUP_FILE>.db <DB_PATH>
sudo chown 10001:10001 <DB_PATH>
sudo chmod 600 <DB_PATH>

sudo docker compose up -d <SERVICE>
sudo docker compose logs --tail=100 <SERVICE>
```

cp remplace la base par la sauvegarde. chown remet le proprietaire numerique utilise par le conteneur. chmod 600 autorise uniquement le proprietaire a lire et ecrire le fichier. Les valeurs UID/GID doivent correspondre a votre Dockerfile / deploiement ; 10001 est un exemple de service non-root.

Attention - Restaurer une sauvegarde V4 alors que le conteneur execute du code exigeant V5 remettra immediatement la base en etat "migration requise". Il faut toujours raisonner ensemble sur la version du code et la version du schema.

15. Permissions Linux utiles autour des fichiers de base

Commande	Effet
ls -lh <chemin>	Liste les fichiers, tailles, proprietaires et permissions sous une forme lisible.
sudo chown 10001:10001 fichier.db	Attribue le fichier a l'UID 10001 et au GID 10001.
sudo chmod 600 fichier.db	Lecture + ecriture pour le proprietaire, aucun droit pour groupe et autres.
sudo chmod 750 dossier	Proprietaire : rwx, groupe : r-x, autres : aucun droit.

Les permissions numeriques se lisent par blocs : 7 = lecture + ecriture + execution, 6 = lecture + ecriture, 5 = lecture + execution, 0 = aucun droit. Pour un fichier SQLite, l'execution n'est pas necessaire.

16. Diagnostic rapide : symptome -> cause probable

Symptome	Verification prioritaire
Discord annonce encore ancienne version attendue	Verifier CURRENT_SCHEMA_VERSION dans le code puis dans le conteneur avec docker exec.

Le code serveur est a jour mais le conteneur ne l'est pas	Rebuild puis up --force-recreate. Utiliser --no-cache si necessaire.
La migration est definie mais non proposee	La constante de version cible n'a peut-etre pas ete incrementee.
Permission denied sur le fichier DB	Verifier ls -lh, proprietaire UID/GID et chmod.
UPDATE modifie 0 ligne	La valeur WHERE n'est pas celle attendue : faire SELECT avant de retenter.
UPDATE modifie plusieurs lignes alors qu'une seule etait attendue	ROLLBACK. Le WHERE est trop large ou l'hypothese sur les donnees est fausse.
Bot toujours en ligne apres stop	Verifier docker compose ps et attendre le delai de presence Discord ; chercher une seconde instance si necessaire.

17. Checklist de securite avant de valider une mise a jour

- La branche active sur le serveur est la bonne.
- git status ne montre pas de modification locale inattendue.
- Le dernier commit correspond au hotfix / release voulu.
- La nouvelle image Docker a ete construite.
- Une sauvegarde existe avant toute migration de schema.
- Le conteneur execute bien la version cible attendue.
- La migration applicative a amene version actuelle = version attendue.
- Une nouvelle sauvegarde existe avant toute correction manuelle de donnees.
- Tout INSERT/UPDATE sensible utilise une transaction et verifie rowcount.
- PRAGMA integrity_check renvoie ok.
- PRAGMA foreign_key_check ne renvoie aucune violation.
- Le conteneur est running apres redemarrage.
- Les logs ne montrent pas d'exception critique.
- Les commandes fonctionnelles et le parcours utilisateur concerne ont ete testes.
- La branche n'est mergee vers main qu'apres validation reelle du deploiement.

Regle finale - Ne jamais accelerer une mise a jour en supprimant les etapes de verification. Les quelques minutes passees a controler branche, image, backup, schema et rowcount sont ce qui rend le deploiement reproductible et recuperable.

18. Aide-memoire de commandes

```
# Git
cd /srv/<SERVER_ID>/apps/<PROJECT>
git status
git branch --show-current
```

```
git log -1 --oneline
git pull --ff-only origin <DEPLOY_BRANCH>

# Docker
cd /srv/<SERVER_ID>/stacks/<PROJECT>
sudo docker compose build
sudo docker compose build --no-cache <SERVICE>
sudo docker compose stop <SERVICE>
sudo docker compose up -d --force-recreate <SERVICE>
sudo docker compose ps
sudo docker compose logs --tail=100 <SERVICE>

# Verifier le code dans le conteneur
sudo docker exec <SERVICE> python -c "from claviger.database.schema import CURRENT_SCHEMA_VERSION;
print(CURRENT_SCHEMA_VERSION)"

# Inspection Linux
ls -lh <chemin>
grep -n "CURRENT_SCHEMA_VERSION" src/claviger/database/schema.py

# Principaux SQL
SELECT column_a, column_b FROM table_name WHERE id = ?;
SELECT COUNT(*) FROM table_name WHERE enabled = ?;
INSERT INTO table_name (column_a, column_b) VALUES (?, ?);
UPDATE table_name SET column_b = ? WHERE id = ?;

# SQLite maintenance
PRAGMA user_version;
PRAGMA table_info(table_name);
PRAGMA integrity_check;
PRAGMA foreign_key_check;
```

Fin du guide